

기초영상처리

추가 영상 처리 내용

외곽선 검출

◆ 외곽선

- ❖ 객체를 둘러 싸고 있는 점들의 조합

◆ 외곽선 검출이란

- ❖ 객체의 외곽선 좌표를 모두 추출하는 작업. Contour tracing
- ❖ 바깥쪽 & 안쪽(홀) 외곽선 → 외곽선의 계층 구조도 표현 가능

◆ 외곽선 객체 하나의 표현 방법

- ❖ `shape = (K,1,2)` (K는 외곽선 좌표 개수)
- ❖ `dtype = numpy.int32`



외곽선 검출

◆ 외곽선 검출 함수

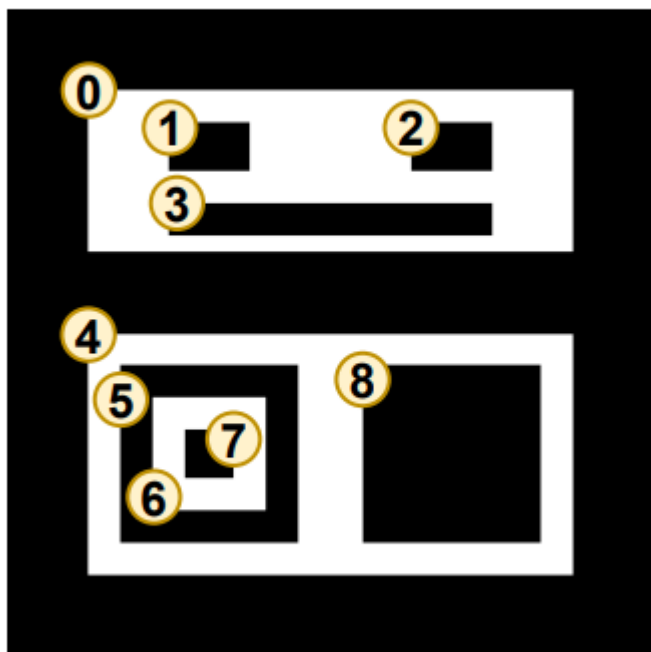
```
cv2.findContours(image, mode, method, contours=None, hierarchy=None,  
                 offset=None) -> contours, hierarchy
```

- image: 입력 영상. non-zero 픽셀을 객체로 간주함.
- mode: 외곽선 검출 모드. cv2.RETR_로 시작하는 상수.
- method: 외곽선 근사화 방법. cv2.CHAIN_APPROX_로 시작하는 상수.
- contours: 검출된 외곽선 좌표. `numpy.ndarray`로 구성된 리스트.
`len(contours)=전체 외곽선 개수(N)`.
`contours[i].shape=(K, 1, 2)`. `contours[i].dtype=numpy.int32`.
- hierarchy: 외곽선 계층 정보. `numpy.ndarray`. `shape=(1, N, 4)`. `dtype=numpy.int32`.
`hierarchy[0, i, 0] ~ hierarchy[0, i, 3]`이 순서대로 next, prev, child, parent
외곽선 인덱스를 가리킴. 해당 외곽선이 없으면 -1.
- offset: 좌표 값 이동 오프셋. 기본값은 (0, 0).

외곽선 검출

◆ mode

• mode:

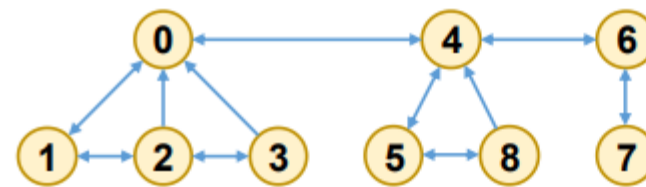


▪ cv2.RETR_EXTERNAL 0 ↔ 4

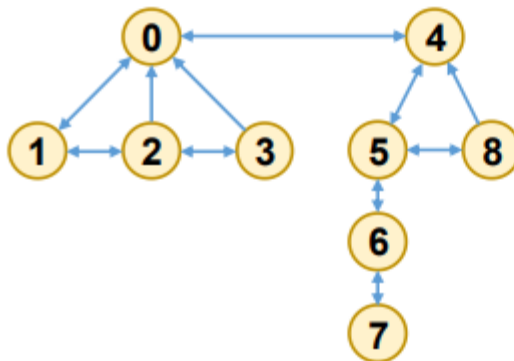
▪ cv2.RETR_LIST



▪ cv2.RETR_CCOMP



▪ cv2.RETR_TREE



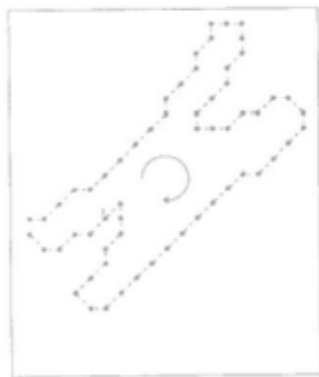
계층 정보 X

계층 정보 O

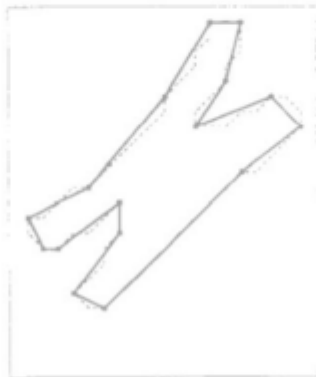
외곽선 검출

◆ method

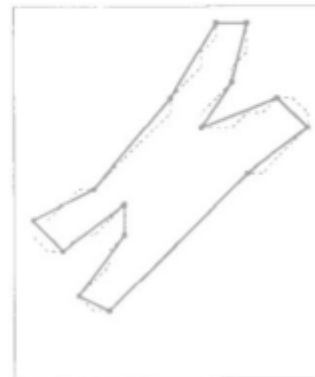
- **cv2.CHAIN_APPROX_NONE** : 근사화 없음
- **cv2.CHAIN_APPROX_SIMPLE** : 수직선, 수평선, 대각선에 대해 끝점만 저장
- **cv2.CHAIN_APPROX_TC89_L1** : Teh & Chin L1 근사화
- **cv2.CHAIN_APPROX_TC89_KCOS** : Teh & Chin k cos 근사화



contours



L1



k cos

외곽선 검출

◆ 외곽선 그리기 함수

```
cv2.drawContours(image, contours, contourIdx, color, thickness=None,  
                 lineType=None, hierarchy=None, maxLevel=None, offset=None)  
-> image
```

- image: 입출력 영상
- contours: (cv2.findContours() 함수로 구한) 외곽선 좌표 정보
- contourIdx: 외곽선 인덱스. 음수(-1)를 지정하면 모든 외곽선을 그린다.
- color: 외곽선 색상
- thickness: 외곽선 두께. thickness < 0이면 내부를 채운다.
- lineType: LINE_4, LINE_8, LINE_AA 중 하나 지정
- hierarchy: 외곽선 계층 정보.
- maxLevel: 그리기를 수행할 최대 외곽선 레벨. maxLevel = 0 이면 contourIdx로 지정된 외곽선만 그린다.

계층 정보를 사용한 외곽선 검출

```
import random
import numpy as np
import cv2

src = cv2.imread("images/contours.bmp", cv2.IMREAD_GRAYSCALE)

if src is None: raise Exception("파일 에러")

contours, hier = cv2.findContours(src, cv2.RETR_CCOMP, cv2.CHAIN_APPROX_NONE)

dst = cv2.cvtColor(src, cv2.COLOR_GRAY2BGR)

idx = 0
while idx >= 0:
    c = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
    cv2.drawContours(dst, contours, idx, c, 2, cv2.LINE_8, hier)
    idx = hier[0, idx, 0]

cv2.imshow('src', src)
cv2.imshow('dst', dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

계측 정보를 사용하지 않은 외곽선 검출

```
import random
import numpy as np
import cv2

src = cv2.imread("images\milkdrop.bmp", cv2.IMREAD_GRAYSCALE)

if src is None: raise Exception("파일 에러")

_, src_bin = cv2.threshold(src, 0, 255, cv2.THRESH_OTSU)

contours, _ = cv2.findContours(src_bin, cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE)

h, w = src.shape[:2]
dst = np.zeros((h, w, 3), np.uint8)

for i in range(len(contours)):
    c = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
    cv2.drawContours(dst, contours, i, c, 1, cv2.LINE_AA)

cv2.imshow('src', src)
cv2.imshow('src_bin', src_bin)
cv2.imshow('dst', dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

다양한 외곽선 처리 함수

◆ 외곽선 처리 함수

함수 이름	설명
<code>cv2.arcLength()</code>	외곽선 길이를 반환
<code>cv2.contourArea()</code>	외곽선이 감싸는 영역의 면적을 반환
<code>cv2.boundingRect()</code>	주어진 점을 감싸는 최소 크기 사각형(바운딩 박스) 반환
<code>cv2.minEnclosingCircle()</code>	주어진 점을 감싸는 최소 크기 원을 반환
<code>cv2.minAreaRect()</code>	주어진 점을 감싸는 최소 크기 회전된 사각형을 반환
<code>cv2.minEnclosingTriangle()</code>	주어진 점을 감싸는 최소 크기 삼각형을 반환
<code>cv2.approxPolyDP()</code>	외곽선을 근사화(단순화)
<code>cv2.fitEllipse()</code>	주어진 점에 적합한 타원을 반환
<code>cv2.fitLine()</code>	주어진 점에 적합한 직선을 반환
<code>cv2.isContourConvex()</code>	컨벡스인지를 검사
<code>cv2.convexHull()</code>	주어진 점으로부터 컨벡스 헐을 반환
<code>cv2.convexityDefects()</code>	주어진 점과 컨벡스 헐로부터 컨벡스 디펙트를 반환

다양한 외곽선 처리 함수

◆ 외곽선 길이 구하기

```
cv2.arcLength(curve, closed) -> retval
```

- curve: 외곽선 좌표. `numpy.ndarray`. `shape=(K, 1, 2)`
- closed: True이면 폐곡선으로 간주
- retval: 외곽선 길이

◆ 면적 구하기

```
cv2.contourArea(contour, oriented=None) -> retval
```

- contour: 외곽선 좌표. `numpy.ndarray`. `shape=(K, 1, 2)`
- oriented: True이면 외곽선 진행 방향에 따라 부호 있는 면적을 반환.
기본값은 False.
- retval: 외곽선으로 구성된 영역의 면적

다양한 외곽선 처리 함수

◆ 바운딩 박스

```
cv2.boundingRect(array) -> retval
```

- array: 외곽선 좌표. `numpy.ndarray`. `shape=(K, 1, 2)`
- retval: 사각형 정보. (x, y, w, h) 튜플.

◆ 바운딩 서클

```
cv2.minEnclosingCircle(points) -> center, radius
```

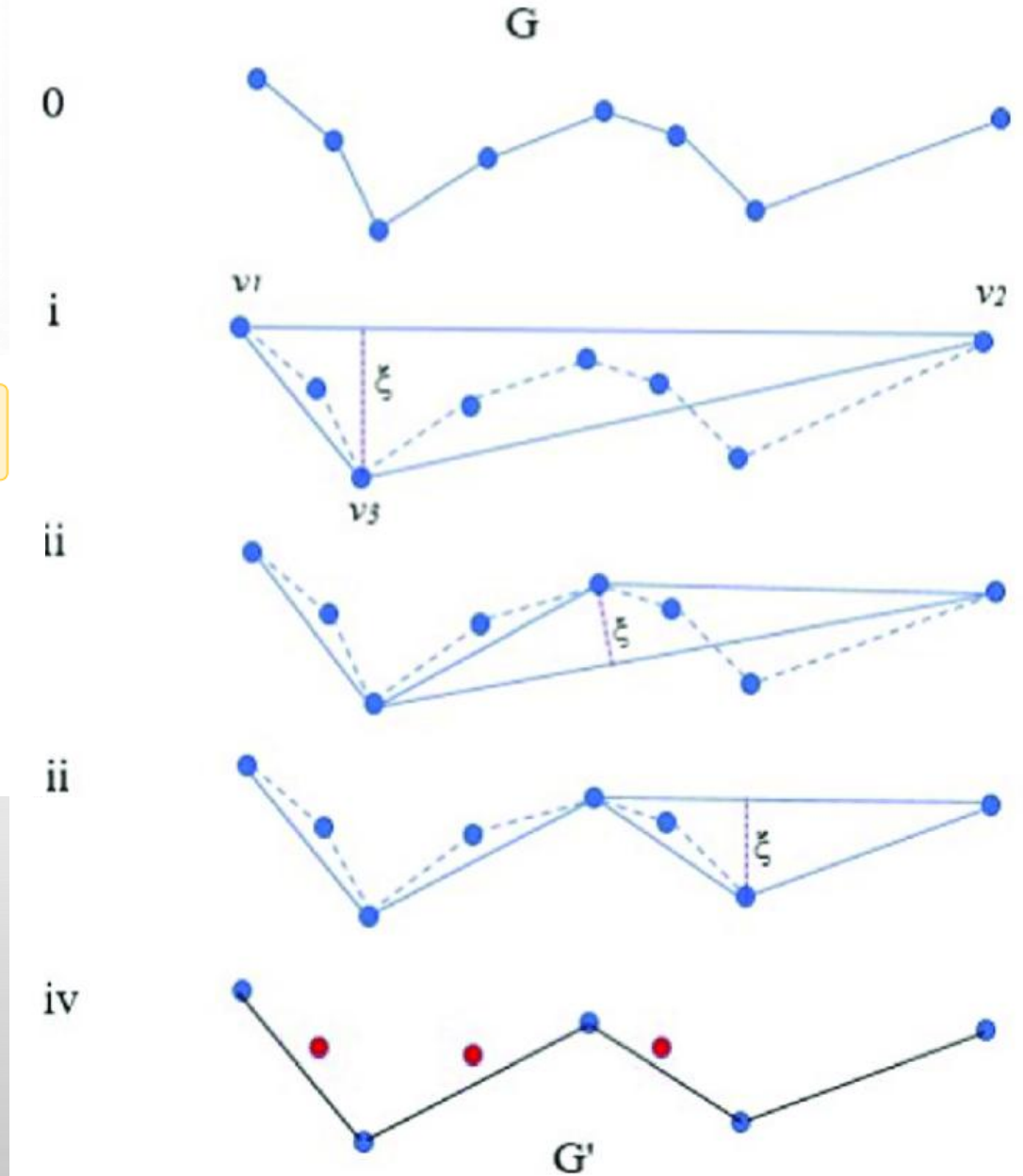
- points: 외곽선 좌표. `numpy.ndarray`. `shape=(K, 1, 2)`
- center: 바운딩 서클 중심 좌표. (x, y) 튜플.
- radius: 바운딩 서클 반지름. 실수.

다양한 외곽선 처리 함수

◆ 외곽선 근사화

```
cv2.approxPolyDP(curve, epsilon, closed, approxCurve=None) -> approxCurve
```

- curve: 입력 곡선 좌표. `numpy.ndarray.shape=(K, 1, 2)`
- epsilon: 근사화 정밀도 조절. 입력 곡선과 근사화 곡선 간의 최대 거리.
e.g) `cv2.arcLength(curve) * 0.02`
- closed: True를 전달하면 폐곡선으로 인식
- approxCurve: 근사화된 곡선 좌표. `numpy.ndarray.shape=(K', 1, 2)`

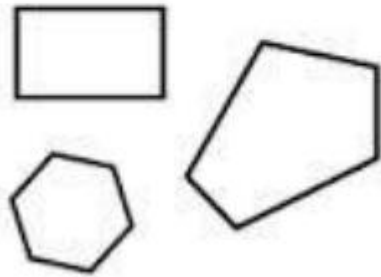


다양한 입력선 처리 함수

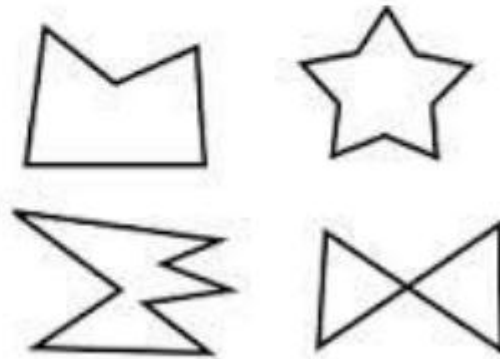
◆ Convex 검사

```
cv2.isContourConvex(contour) -> retval
```

- contour: 입력 곡선 좌표. `numpy.ndarray`. `shape=(K, 1, 2)`
- retval: 컨벡스이면 True, 아니면 False.



Convex Polygons



Non-convex Polygons

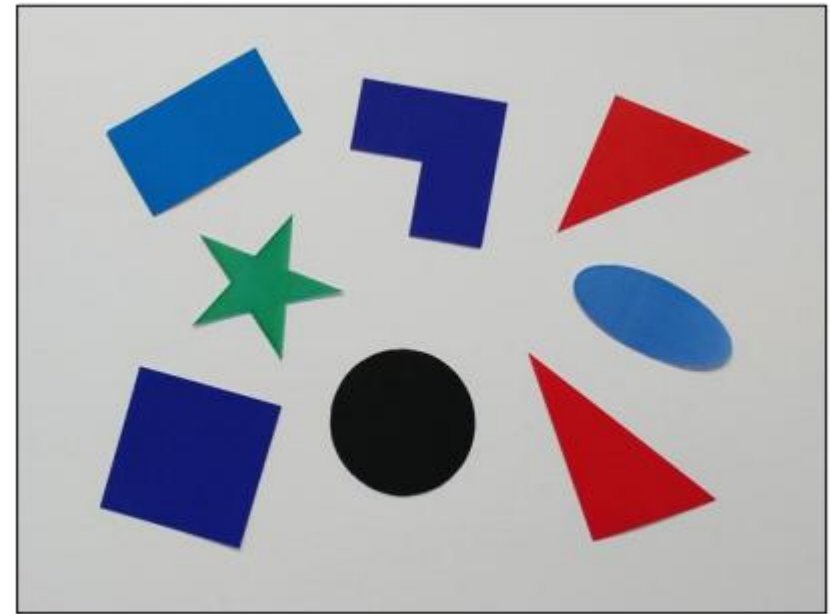
다각형 검출 프로그램 만들기

◆ 다각형 판별 프로그램

❖ 다양한 다각형 객체 영상에서 삼각형, 사각형, 원 찾기

◆ 구현 순서

1. 이진화
2. 외곽선 찾기
3. 외곽선 근사화
4. 너무 작은 객체와 컨벡스가 아닌 객체 제외
5. 꼭지점 개수 확인



다각형 검출 프로그램 만들기

◆ 원 판별하기

❖ 정해진 외곽선 길이에 대한 넓이 비율이 가장 큰 형태가 원

- 넓이(A)와 외곽선 길이(P)의 비율을 검사

$$\frac{A}{P} = \frac{\pi r^2}{2\pi r} \rightarrow \frac{A}{P^2} = \frac{\pi r^2}{4\pi^2 r^2} = \frac{1}{4\pi} \rightarrow 4\pi \frac{A}{P^2} = 1$$

- 1에 가까울수록 원으로 판단

```
length = cv2.arcLength(pts, True)
area = cv2.contourArea(pts)
ratio = 4. * math.pi * area / (length * length)

if ratio > 0.85:
    setLabel(img, pts, 'CIR') # 원
```

```
import math
import cv2

def setLabel(img, pts, label):
    (x, y, w, h) = cv2.boundingRect(pts)
    pt1 = (x, y)
    pt2 = (x + w, y + h)
    cv2.rectangle(img, pt1, pt2, (0, 0, 255), 1)
    cv2.putText(img, label, pt1, cv2.FONT_HERSHEY_PLAIN, 1, (0, 0, 255))

img = cv2.imread("images/polygon.bmp", cv2.IMREAD_COLOR)
if img is None: raise Exception("파일 에러")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, img_bin = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)
contours, _ = cv2.findContours(img_bin, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)

for pts in contours:
    if cv2.contourArea(pts) < 400: # 너무 작으면 무시
        continue
    approx = cv2.approxPolyDP(pts, cv2.arcLength(pts, True)*0.02, True)
    vtc = len(approx)

    if vtc == 3:
        setLabel(img, pts, 'TRI')
    elif vtc == 4:
        setLabel(img, pts, 'RECT')
    else:
        length = cv2.arcLength(pts, True)
        area = cv2.contourArea(pts)
        ratio = 4. * math.pi * area / (length * length)

        if ratio > 0.85:
            setLabel(img, pts, 'CIR')
cv2.imshow('img', img)
cv2.waitKey()
cv2.destroyAllWindows()
```